QSWAP

Security Assessment and Code Audit

Mundus

Feb 17th 2026
Final Report

INTRODUCTION

QSwap is a Uniswap V2-style Automated Market Maker (AMM) DEX for the Qubic blockchain. It enables users to create liquidity pools for QU/Asset pairs, provide liquidity, and swap between QU and various assets.

The contract implements constant product market making ($x \cdot y = k$) with a 0.3% swap fee distributed among

- Shareholders (27%)
- QX (5%)
- Invest & Rewards (3%)
- Burn (1%)
- Liquidity providers (64%)

AUDIT PROCESS

Following considerations have been tested and reviewed by Mundus team:

- Reviewing google test files that are written by SC developer.
- Trying several attack vectors to exploit the SC.
- Ensuring contract logic meets the specifications.
- Thorough line-by-line manual review.

RESULTS

We identified a range of security issues in our assessment, from **critical** to minor. We recommend fixing them to meet security standards and best practices.

OVERVIEW

1. Project name: **QSWAP**
2. Platform: **QUBIC SC system**
3. Is new SC deployment: **NO**
4. Is upgraded SC: **YES**
5. Project size: Large
6. List of github commits to review:

Rev 1	https://github.com/qubic/core/tree/v1.271.0
Rev 2	57d24af49c3680b2f54e39231e8ed7c721465929 115ca4744e037b992f89776984427df09d8a84af
Rev 3	cb9a4fac88d5784da78a1cede29aa3472c37c1cd

RESULTS SUMMARY

Total Issues	16
Critical	1
Major	0
Medium	2
Minor	13

AUDIT SCOPE

Rev 1

Filename	SHA	Latest commit
Qswap.h	466767b0346b2006f36aee7a367180aaaed2d85b	fix QSwap
contract_qswap.cpp (gtest)	698f36b5cdc7c1b9fabc42326c166525191ee72f	fix qswap test and adjust epoch for begin_epoch proc

Rev 2

Filename	SHA	Latest commit
Qswap.h	57d24af49c3680b2f54e39231e8ed7c721465929	epoch -> tick
contract_qswap.cpp (gtest)	115ca4744e037b992f89776984427df09d8a84af	fix unit tests

Rev 3

Filename	SHA	Latest commit
Qswap.h	cb9a4fac88d5784da78a1cede29aa3472c37c1cd	QSwap audit fixes
contract_qswap.cpp (gtest)	cb9a4fac88d5784da78a1cede29aa3472c37c1cd	QSwap audit fixes

STATUS OF FINDINGS IN REV 1

ID	Title	Category	Severity	Status
QSWAP-01	TransferShareManagementRights Fee Handling Issues	Logic	Medium	Fixed
QSWAP-02	CreatePool Only Allows Asset Issuer Despite Different Input	Logic	Minor	Not Fixed
QSWAP-03	No Fee Rate Validation	Logic	Minor	Fixed
QSWAP-04	Potential Reserve Underflow in Pool State Updates	Logic	Minor	Fixed
QSWAP-05	Redundant/Duplicate Code in SwapQuForExactAsset	Logic	Minor	Fixed
QSWAP-06	Typo in Comments	Code Convention	Minor	Fixed
QSWAP-07	Failed unit test: QuoteTest	Logic	Minor	Fixed
QSWAP-08	Failed unit test: SwapAssetForExactQu	Logic	Minor	Fixed
QSWAP-9	Slow traversing for the array `state.mPoolBasicStates`	Logic	Minor	Acknowledged - Not Fixed
QSWAP-10	Inefficient Pool Slot Recycling	Logic	Minor	Acknowledged - Not Fixed

STATUS OF FINDINGS IN REV 2

ID	Title	Category	Severity	Status
QSWAP-11	Wasteful Cross-Contract Calls for Static Fees	Logic	Minor	Fixed
QSWAP-12	Inefficient TransferShareManagementRights Logic	Logic	Minor	Fixed
QSWAP-13	Slow Square Root Algorithm	Logic	Minor	Fixed

STATUS OF FINDINGS IN REV 3

ID	Title	Category	Severity	Status
QSWAP-14	Pool Creation Front-Running / ID Spoofin	Logic	Critical	Fixed
QSWAP-15	"Free Option" / Dust Attack via Rounding	Logic	Medium	Fixed
QSWAP-16	Forbidden operator :: used	Code Convention	Minor	Acknowledged - Not Fixed

FINDINGS

QSWAP-01 TransferShareManagementRights Fee Handling Issues

- File(s) affected: Qswap.h:2244-2282
- Description: The procedure has several issues
  - If fee is insufficient, function returns without refund as at lines 2246-2248.
  - If successful, the fee taken doesn't update to any tracked state variable.
  - The SC always assumes the destination contract always charges 100 Qus, while not all SCs accept fees for Management Right Transfer.
- Impact:
  - Users lost funds due to no refund.
  - Dividends will be incorrect due to missed tracking fees. The retained fees become "zombie funds"—permanently locked in the contract's balance, never distributed to shareholders, or never burned.
  - Users are overcharged for transfers to free contracts.
- Recommendation:
  - Add refund on insufficient fee.
  - Update taken fee to track state variables.
  - Do not assume the fee is 100 Qus. Capture the return value of `qpi.releaseShares` (which indicates the actual fee paid) and use that value to calculate the correct refund for the user.

QSWAP-02 CreatePool Only Allows Asset Issuer Despite Different Input

- **File(s) affected:** Qswap.h:956-960
- **Description:** The CreatePool procedure accepts **assetIssuer** as input but completely ignores it. Instead, it only checks if the invocator issued the asset as at line 956 of Qswap.h.
- **Impact:**
  - Only asset issuers can create pools for their own assets
  - Third parties cannot create pools for popular assets
  - This severely limits DEX functionality
- **Recommendation:** If this is intentional as we only limit the asset issuer to create pools, we should remove the redundant `assetIssuer` from `CreatePool_input`, and only use the `invocator` in the procedure `CreatePool` instead.

QSWAP-03 No Fee Rate Validation

- **File(s) affected:** Qswap.h:2312-2319, 2330-2334
- **Description:** Fee rates are set directly without validation. If rates were accidentally set incorrectly (e.g., during an upgrade), it could break the AMM math.
- **Recommendation:** Add validation constants and checks, example:
  - ASSERT(state.swapFeeRate < QSWAP_SWAP_FEE_BASE);
  - ASSERT(state.shareholderFeeRate + state.investRewardsFeeRate + state.qxFeeRate + state.burnFeeRate <= 100);

QSWAP-04 Potential Reserve Underflow in Pool State Updates

- File(s) affected: Qswap.h:1615, 1776, 1963-1967, 2147-2152
- Description: Multiple swap functions subtract from reservedQuAmount without explicit underflow checks. The AMM math should prevent this in normal operation, however edge cases with rounding could cause issues.
- Recommendation: Use saturating subtraction or add explicit checks, example:

```
locals.totalDeduction = locals.quAmountOut + sint64(locals.feeToShareholders.low)
+ ...;
if (locals.poolBasicState.reservedQuAmount < locals.totalDeduction)
{
    // Handle error
    return;
}
locals.poolBasicState.reservedQuAmount -= locals.totalDeduction;
```

QSWAP-05 Redundant/Duplicate Code in SwapQuForExactAsset

- File(s) affected: Qswap.h:1718-1729
- Description: The same check appears twice
- Impact: Wasted computation, maintenance burden
- Recommendation:

QSWAP-06 Typo in Comments

- File(s) affected: Qswap.h:632, 690, 753, 822, 986
- Description: "solt" should be "slot"
- Recommendation:

QSWAP-07 Failed unit test: QuoteTest

- **File(s) affected:** contract_qswap.cpp:314
- **Description:** When calculating the division in `getAmountInTakeFeeFromInToken`, the developer intentionally added 1 for the output, but forgot to do the same for the unit test.
- **Recommendation:** Increase 1 for the unit test expected value at line contract_qswap.cpp:314

QSWAP-08 Failed unit test: SwapAssetForExactQu

- **File(s) affected:** contract_qswap.cpp:588
- **Description:** When calculating the division in `getAmountInTakeFeeFromOutToken`, the developer intentionally added 1 for the output, but forgot to do the same for the unit test.
- **Recommendation:** Increase 1 for the unit test expected value at line contract_qswap.cpp:588

QSWAP-9 Slow traversing for the array **`state.mPoolBasicStates`**

- **File(s) affected:** Qswap.h:331
- **Description:** Every time a user wants to Swap, Add Liquidity, or just check a price, the contractLinear performs a scanning (O(N)) for **`state.mPoolBasicStates`**.
- **Impact:** This affects Compute Tick Limits and Scalability.
- **Recommendation:** Use QPI HashMap object with Hash Map Lookup (O(1)) instead of using the Array object for `state.mPoolBasicStates`.

```
HashMap<Asset, PoolBasicState, QSWAP_MAX_POOL>
mPoolBasicStates;
```

QSWAP-10 Inefficient Pool Slot Recycling

- **File(s) affected:** Qswap.h:1358-1482
- **Description:** The contract uses a fixed-size array `mPoolBasicStates` to track pools. Although there is a low chance this array will be full, it is recommended to have a mechanism to reuse unused pools.
- **Impact:** This affects the slot to remain technically "occupied" even though the pool is empty for a long time with no activity, wasting the array `mPoolBasicStates`.
- **Recommendation:** Should add a mechanism to auto-remove 0 total liquidity after a long time (ie. after a threshold of epochs) with no activity.

QSWAP-11 Wasteful Cross-Contract Calls for Static Fees

- **File(s) affected:** Qswap.h:2189, 514, 869, 942
- **Description:** The contract calls the external contract `QX` to fetch fee settings in every execution of critical procedures and functions (Fees, IssueAsset, CreatePool, TransferShareOwnershipAndPossession) using `CALL_OTHER_CONTRACT_FUNCTION`.
- **Impact:** Inter-contract calls (`CALL_OTHER_CONTRACT_FUNCTION`) are expensive operations. Fee settings (like `assetIssuanceFee`) change very rarely, making this redundant overhead for every user transaction.
- **Recommendation:** Cache the QX fees in `QSWAP`'s state variables (e.g., `state.cachedIssuanceFee`). Update the new fees at the beginning of each epoch.

QSWAP-12 Inefficient `TransferShareManagementRights` Logic

- **File(s) affected:** Qswap.h:2267-2336
- **Description:** The procedure attempts to release shares with a 0 fee first to "probe" the required fee, fails, and then calls it again with the correct fee.
- **Impact:** Calling a system procedure twice doubles the overhead.
- **Recommendation:** Call `qpi.releaseShares` once, passing the full `qpi.invocationReward()` (or `locals.reward`) as the `offeredTransferFee`. The system will automatically deduct only the required fee and return the success status.

QSWAP-13 Slow Square Root Algorithm

- **File(s) affected:** Qswap.h:354-375
- **Description:** The `sqrt` function uses a division-heavy iterative method inside `AddLiquidity`.
- **Impact:** `div` (division) is CPU-intensive. For large liquidity values, this loop can run many times, consuming significant ticks.
- **Recommendation:** Replace with a bitwise integer square root algorithm, which relies on bit-shifts and additions and is significantly faster for the `uint128` data type involved here.

QSWAP-14 Pool Creation Front-Running / ID Spoofing

- File(s) affected: Qswap.h:933-1001
- Description: The `CreatePool` function suffers from a critical logic error where it validates the **caller** but trusts the **input** for the pool's identity.
  - **The Check:** The contract verifies if the *caller* (Invocator) has issued an asset with the requested `assetName`.
  - **The Flaw:** When creating the pool, it uses the `assetIssuer` provided in the *user input*, rather than the authenticated *Invocator's ID*.
  - **The Exploit Steps**:
    i. **Setup:** The Attacker issues a worthless token with the **same name** (e.g., "COIN") as a legitimate Victim's upcoming token.
    ii. **Bypass:** The Attacker calls `CreatePool`. The contract checks if the Attacker owns "COIN". Since they issued a dummy version, this check **passes**.
    iii. **Spoofing:** In the input parameters, the Attacker sets the `assetIssuer` to the **Victim's ID**.
    iv. **Result:** The contract creates a pool registered under the Victim's ID (`VictimID + "COIN"`).
- Impact: This vulnerability enables three distinct attacks:
  - **Denial of Service (DoS):** The contract does not allow duplicate pools. Since the slot `VictimID + "COIN"` is now occupied by the attacker, the legitimate issuer (Victim) cannot create their official pool.
  - **Phishing / Trust Exploitation:** Users and trading bots rely on the Issuer ID to verify tokens. Because the pool is registered with the correct (Victim's) ID, it appears official. Users may blindly trust the pool is safe.
  - **Financial Rug Pull:** If the Attacker holds *any* amount of the Victim's real token, they can become the first liquidity provider for this "official" pool. They can set a manipulated price (e.g., extremely high QUBIC cost per COIN), wait for bots/users to buy in, and then withdraw all liquidity, stealing the funds.
- Recommendation: In `CreatePool`, enforce that the `input.assetIssuer` matches the invocator, or simply ignore the input and use the invocator's ID.

```
// Fix in CreatePool
if (input.assetIssuer != qpi.invocator()) {
     // Reject or overwrite
     return;
}
// OR simply remove the assetIssuer from input struct
```

- **Re**-Note: **This issue was originally reported in the Rev 1 audit but remains unaddressed. While the rationale for retaining the current logic is unclear, the security implication is critical: the vulnerability allows any actor (again, ANY ACTOR) to compromise the Smart Contract (SC). This functions effectively as a backdoor for malicious exploitation. We strongly advise immediate action.**

QSWAP-15 "Free Option" / Dust Attack via Rounding

- **File(s) affected:** Qswap.h:1501-1626
- **Description:** The fee calculation logic allows for fee-free trading on small amounts due to integer truncation, found in Fee calculations in `SwapExactQuForAsset`:

```
locals.swapFee = div(uint128(locals.quAmountIn) *
      uint128(state.swapFeeRate), uint128(QSWAP_SWAP_FEE_BASE));
```

  With a `swapFeeRate` of 30 (0.3%) and a base of 10,000, any `quAmountIn` less than 334 results in a `swapFee` of 0.
- **Impact:** A bot could execute thousands of micro-trades to drain liquidity without paying fees, or manipulate prices at no cost (other than execution limits).
- **Recommendation:** Impose a minimum fee for any amount less than 334, ie. 10 QUs.

QSWAP-16 Forbidden operator :: used

- **File(s) affected:** Qswap.h:2341-2342
- **Description:** Forbidden operator :: is used in the code.
- **Recommendation:** Remove the operator to comply with the Qubic SC development.